PDF Forge

Backend Development Plan

Version 1.0 | October 2026

Stack: Node.js + Express + MongoDB

github.com/MuhammadSaqibIslam/PDF-Converter

1. Project Overview

PDF Forge is a browser-based PDF/Image/Dev tools website. Currently it runs entirely on the client-side with no backend. This plan outlines the backend architecture needed to add user accounts, cloud processing, file storage, payments, and admin functionality.

Current State: 42+ tool pages, vanilla JS frontend, all processing via pdf-lib, PDF.js, Canvas API, and Web Crypto API in the browser.

2. Tech Stack

Runtime & Framework

- Node.js (v20 LTS) + Express.js
- TypeScript for type safety
- PM2 for process management in production

Database

- MongoDB Atlas (primary) - users, subscriptions, file metadata
- Redis - caching, session store, rate limiting, job queues

File Storage

- AWS S3 / Cloudflare R2 - uploaded files & processed outputs
- Signed URLs for secure file access (15 min expiry)
- Auto-delete processed files after 1 hour

Authentication

- JWT (access token: 15min, refresh token: 7 days)
- bcrypt for password hashing
- Google & GitHub OAuth 2.0

Payments

- Stripe - subscriptions & one-time payments
- Webhook-based payment verification

Deployment

- Docker containers on Railway / Render / VPS
- Nginx reverse proxy + SSL (Lets Encrypt)
- GitHub Actions CI/CD pipeline

3. Backend Folder Structure

```
backend/
|-- src/
|   |-- config/
|   |   |-- db.ts            (MongoDB connection)
|   |   |-- redis.ts         (Redis connection)
|   |   |-- s3.ts            (S3/R2 client)
|   |   |-- stripe.ts        (Stripe config)
|   |   |-- env.ts           (Environment variables)
|   |-- models/
|   |   |-- User.ts
|   |   |-- File.ts
|   |   |-- Subscription.ts
|   |   |-- ToolUsage.ts
|   |-- routes/
|   |   |-- auth.routes.ts
|   |   |-- user.routes.ts
|   |   |-- file.routes.ts
|   |   |-- tool.routes.ts
|   |   |-- payment.routes.ts
|   |   |-- admin.routes.ts
|   |-- controllers/
|   |   |-- auth.controller.ts
|   |   |-- user.controller.ts
|   |   |-- file.controller.ts
|   |   |-- tool.controller.ts
|   |   |-- payment.controller.ts
|   |   |-- admin.controller.ts
|   |-- middleware/
|   |   |-- auth.ts          (JWT verification)
|   |   |-- rateLimiter.ts   (Rate limiting)
|   |   |-- upload.ts        (Multer file upload)
|   |   |-- validate.ts      (Request validation)
|   |   |-- errorHandler.ts  (Global error handler)
|   |-- services/
|   |   |-- email.service.ts  (Nodemailer/Resend)
|   |   |-- s3.service.ts     (File upload/download)
|   |   |-- stripe.service.ts (Payment logic)
|   |   |-- pdf.service.ts    (Server-side PDF ops)
|   |-- jobs/
|   |   |-- cleanupFiles.ts   (Delete expired files)
|   |   |-- usageReport.ts    (Daily stats)
|   |-- utils/
|   |   |-- logger.ts        (Winston logger)
|   |   |-- helpers.ts
|   |-- app.ts               (Express app setup)
|   |-- server.ts            (Entry point)
|-- .env.example
|-- package.json
|-- tsconfig.json
|-- Dockerfile
```

4. Database Schema (MongoDB)

Users Collection

```
{
  _id: ObjectId,
  name: String,
  email: String (unique),
  password: String (hashed),
  avatar: String (URL),
  provider: "local" | "google" | "github",
  plan: "free" | "pro" | "premium",
  stripeCustomerId: String,
  toolsUsedToday: Number,
  dailyLimit: Number (free:5, pro:50, premium:unlimited),
  isVerified: Boolean,
  isAdmin: Boolean,
  createdAt: Date, updatedAt: Date
}
```

Files Collection

```
{
  _id: ObjectId,
  userId: ObjectId (ref: Users),
  originalName: String,
  s3Key: String,
  size: Number (bytes),
  mimeType: String,
  toolUsed: String,
  status: "uploaded" | "processing" | "done" | "failed",
  outputKey: String,
  expiresAt: Date (TTL index),
  createdAt: Date
}
```

Subscriptions Collection

```
{
  _id: ObjectId,
  userId: ObjectId (ref: Users),
  stripeSubId: String,
  plan: "pro" | "premium",
  status: "active" | "canceled" | "past_due",
  currentPeriodStart: Date,
  currentPeriodEnd: Date,
  cancelAtPeriodEnd: Boolean,
  createdAt: Date
}
```

ToolUsage Collection (Analytics)

```
{
  _id: ObjectId,
  userId: ObjectId (nullable),
  tool: String (e.g. "merge-pdf"),
  fileSize: Number,
  duration: Number (ms),
  status: "success" | "failed",
  ip: String (hashed),
```

```
  createdAt: Date
}
```

5. API Endpoints

Authentication /api/auth/

- POST /register - Sign up with email/password
- POST /login - Login, returns JWT tokens
- POST /refresh - Refresh access token
- POST /logout - Invalidate refresh token
- POST /forgot-password - Send reset email
- POST /reset-password/:token - Reset password
- GET /google - Google OAuth redirect
- GET /google/callback - Google OAuth callback
- GET /github - GitHub OAuth redirect
- GET /github/callback - GitHub OAuth callback
- GET /verify-email/:token - Email verification

User /api/user/

- GET /me - Get current user profile
- PUT /me - Update profile (name, avatar)
- PUT /me/password - Change password
- DELETE /me - Delete account + all data
- GET /me/usage - Get todays usage stats
- GET /me/history - File processing history

File Operations /api/files/

- POST /upload - Upload file(s) to S3, get file ID
- GET /download/:id - Get signed download URL
- DELETE /:id - Delete a file
- GET /my-files - List users files (paginated)

Tools /api/tools/

- POST /merge-pdf - Merge multiple PDFs
- POST /split-pdf - Split PDF by pages
- POST /compress-pdf - Compress PDF
- POST /pdf-to-word - Convert PDF to DOCX
- POST /pdf-to-image - Convert PDF to PNG/JPG
- POST /image-to-pdf - Convert images to PDF
- POST /rotate-pdf - Rotate PDF pages
- POST /watermark - Add watermark to PDF
- POST /protect-pdf - Add password to PDF
- POST /unlock-pdf - Remove PDF password

Payments /api/payments/

- POST /create-checkout - Create Stripe checkout session
- POST /webhook - Stripe webhook handler
- GET /plans - Get available plans & prices
- POST /cancel - Cancel subscription
- GET /billing - Get billing history & invoices

Admin /api/admin/ (admin only)

- GET /stats - Dashboard stats (users, revenue, usage)
- GET /users - List all users (paginated, searchable)
- PUT /users/:id - Update user (ban, change plan)
- GET /tools/stats - Tool usage analytics

6. Rate Limiting & Plans

Plan Limits

```
FREE Plan:
  - 5 tool uses per day
  - Max file size: 10 MB
  - No cloud storage
  - Basic tools only

PRO Plan ($4.99/month):
  - 50 tool uses per day
  - Max file size: 50 MB
  - 1 GB cloud storage
  - All tools + priority processing
  - File history (7 days)

PREMIUM Plan ($9.99/month):
  - Unlimited tool uses
  - Max file size: 200 MB
  - 10 GB cloud storage
  - All tools + priority processing
  - File history (30 days)
  - API access (coming soon)
```

Rate Limiting Rules

- Guest (no account): 3 requests/day per IP
- Free users: 5 tool uses/day, 20 API calls/min
- Pro users: 50 tool uses/day, 60 API calls/min
- Premium users: Unlimited tools, 120 API calls/min
- Auth endpoints: 5 attempts/15 min (brute force protection)

7. Security

- JWT tokens with short expiry (15 min access, 7 day refresh)
- bcrypt password hashing (12 salt rounds)
- Helmet.js for HTTP security headers
- CORS configured for frontend domain only
- Input validation with Joi/Zod on all endpoints
- File type validation (magic bytes, not just extension)
- NoSQL injection protection
- XSS protection via sanitization
- Rate limiting on all endpoints (express-rate-limit + Redis)
- Signed S3 URLs with expiry (no direct file access)
- HTTPS only in production
- Environment variables for all secrets (.env)
- Request logging with Winston (no sensitive data)

8. Email System

Transactional Emails (via Resend / Nodemailer)

- Welcome email after registration
- Email verification link
- Password reset link (expires in 1 hour)
- Subscription confirmation
- Subscription cancellation
- Payment failed notification
- Weekly usage summary (optional)

9. Background Jobs (Bull + Redis)

Scheduled Jobs

- cleanupFiles - Every hour, delete expired files from S3
- usageReport - Daily at midnight, generate usage stats
- resetDailyLimits - Daily at midnight, reset counters
- expiredSubscriptions - Every 6 hours, check expired subs

Queue Jobs (On Demand)

- processFile - Heavy PDF processing (merge, convert, compress)
- sendEmail - Email sending queue (retry on failure)
- generateThumbnail - Create file preview thumbnails

10. Development Phases

Phase 1 - Foundation (Week 1-2)

- Project setup (Express + TypeScript + MongoDB)
- User authentication (register, login, JWT, OAuth)
- Email verification & password reset
- Basic middleware (auth, error handler, validation)

Phase 2 - File System (Week 3-4)

- S3 file upload/download with signed URLs
- File processing endpoints (merge, split, compress)
- Background job processing with Bull queues
- Auto-cleanup of expired files

Phase 3 - Monetization (Week 5-6)

- Stripe integration (checkout, webhooks)
- Subscription plans (free, pro, premium)
- Rate limiting per plan
- Billing history & invoice access

Phase 4 - Admin & Polish (Week 7-8)

- Admin dashboard API (stats, user management)
- Tool usage analytics
- Performance optimization & caching
- Docker setup & deployment pipeline
- Load testing & security audit

11. Environment Variables (.env)

```
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/pdfforge
REDIS_URL=redis://localhost:6379
JWT_ACCESS_SECRET=your-access-secret
JWT_REFRESH_SECRET=your-refresh-secret
S3_BUCKET=pdfforge-files
S3_REGION=auto
S3_ACCESS_KEY=your-access-key
S3_SECRET_KEY=your-secret-key
STRIPE_SECRET_KEY=sk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx
GOOGLE_CLIENT_ID=xxx
GITHUB_CLIENT_ID=xxx
RESEND_API_KEY=re_xxx
EMAIL_FROM=noreply@pdfforge.com
FRONTEND_URL=https://pdfforge.com
```

12. Quick Start Commands

```
npm install          # Install dependencies
npm run dev          # Development server
```

```
npm run build        # Build for production
npm start            # Start production
docker-compose up -d # Run with Docker
npm test             # Run tests
```